VENDOR ADAPTER RESEARCH — WIRELESS SESSION DATA

Getting client-session data out of six WiFi vendors (and Meraki, separately)

For every vendor: how you extract client/session data, what format it's in, whether it covers the four fields `SessionEvent` needs — identity, location, timestamp, session state — and what it costs to get access and test against it.

Compiled 2026-08-07 **Method** 7 parallel research passes over each vendor's official developer docs

Target model `SessionEvent(user_id, device_mac, location_id, event_type, timestamp, source, raw)`

JUMP TO

Executive summary

Primary way to get data out, its wire format, how cleanly the four `SessionEvent` fields map, and what it takes to get credentials and a place to test.

VENDOR	PRIMARY EXTRACTION PATH(S)	FORMAT	4 CORE FIELDS
Cisco Catalyst / ISE / DNAC / CMX	RADIUS accounting (ISE) · pxGrid session directory · DNAC/Catalyst Center Assurance REST · CMX/DNA Spaces location	RADIUS AVPs · JSON/REST · JSON over STOMP (pxGrid)	ID LOC TS ST
Meraki cloud-managed, separate stack	Dashboard REST API (poll) · webhooks (`client_connectivity`) · Scanning/Location API v3 ·	JSON over HTTPS	ID LOC TS ST

VENDOR	PRIMARY EXTRACTION PATH(S)	FORMAT	4 CORE FIELDS
	RADIUS via splash-page flow only		
Aruba HPE — Central / ClearPass	ClearPass RADIUS accounting · ClearPass REST (Insight/GuestManager) · Central Monitoring REST (poll) · Central Streaming API (WSS)	RADIUS AVPs · JSON/REST · JSON over WSS	ID LOC TS ST
Ruckus CommScope — SmartZone / RUCKUS One	RADIUS accounting (via SmartZone) · SmartZone Northbound REST (poll) · Northbound Data Streaming (GPB/MQTT) · RUCKUS One cloud REST	RADIUS AVPs · JSON/REST · Protobuf over MQTT	ID LOC TS ST
Ubiquiti UniFi Network / Site Manager	Official UniFi Network API (REST) · legacy `/stat/sta` + `/stat/session` (unofficial) · WebSocket events (unofficial) · RADIUS via external server	JSON/REST · JSON over WS · RADIUS AVPs	ID LOC TS ST
Juniper Mist cloud-managed, AI-driven	RADIUS accounting (Access Assurance) · REST (poll) · WebSocket streaming · webhooks (`client-join`, `client-sessions`)	RADIUS AVPs · JSON/REST · JSON over WS	ID LOC TS ST
Huawei AirEngine / iMaster NCE-Campus	RADIUS accounting (AirEngine/WAC) · NCE-Campus Northbound REST (poll, Terminal Mgmt API) · eSight NBI (legacy)	RADIUS AVPs · JSON/REST	ID LOC TS ST

✓ confirmed / clean   △ partial / conditional   ✕ weak or unconfirmed

Cross-vendor pattern

RADIUS accounting is the actual common denominator — not each vendor's REST API

Every one of the seven stacks supports standard RADIUS accounting (`Acct-Status-Type` : Start / Interim-Update / Stop), and in every case it's the *cleanest* match to `SessionEvent` 's `event_type` enum and to a real, authenticated `user_id` (the RADIUS `User-Name` attribute). This validates the presence-platform's FreeRADIUS-first design: it's the one mechanism that's vendor-neutral by construction, requires no vendor SDK, and doesn't get relicensed or renamed under you.

The vendor-native REST/streaming/cloud APIs (DNAC, Central, SmartZone NBI, UniFi Network API, Mist, NCE-Campus) are complementary, not substitutes — they're usually where you get a *human-readable* `location_id` (AP name, floor, site) instead of a raw NAS-IP or AP MAC, and richer x/y location. But most of them are **poll-based snapshots** of "current clients," not a session-lifecycle event stream, so start/update/stop has to be inferred by diffing polls unless you also wire up a streaming/webhook channel (pxGrid, Central Streaming, Mist webhooks, Ruckus GPB/MQTT).

Identity is the field that actually breaks by default on five of the seven stacks (Meraki, Ruckus, UniFi, Mist, Huawei): open or PSK networks give you a device MAC and an SSID, not a person. A real `user_id` only shows up once 802.1X, a captive/splash portal, or a NAC add-on (Mist Access Assurance, ClearPass) is in the picture — which again routes back to RADIUS as the thing actually carrying identity. If the adapter's job is producing a trustworthy `user_id`, RADIUS accounting is doing most of the real work industry-wide, and the vendor REST APIs are enrichment.

Ease of building & testing an adapter, ranked

Based on whether a free/self-serve sandbox exists, or real licensed hardware is required.

1 **1. Cisco** — Best

Free DevNet account gives both Always-On and reservable ISE/Catalyst Center sandboxes, including an ISE + RADIUS-simulator sandbox — no hardware purchase.

2 **2. Meraki** — Free sandbox

Always-on developer sandbox org for API/dashboard mechanics; doesn't generate organic client traffic, so live session flows still need a real network.

3 **3. Ruckus (RUCKUS One)** — Free trial

60-day cloud trial gives real API access; SmartZone/NBI and SPoT still need owned/licensed controller hardware.

4 **4. Aruba** — Partially gated

Central had a historical 90-day trial, but the Streaming API needs an Advanced-licensed device and API Gateway enabled by an HPE rep; ClearPass has no public trial.

5 **5. Juniper Mist** Unclear

Requires a cloud org account and API token; no public free-trial/sandbox org was confirmed — likely a partner or demo account.

6 **6. Ubiquiti / UniFi** No sandbox

No Ubiquiti-hosted sandbox exists at all, official or otherwise — every path requires an owned/adopted UniFi console.

7 **7. Huawei** Hardest

Most official docs render blank without a support.huawei.com login; NBI access needs an admin-provisioned account on a live controller; real testing likely needs a partner/ESDP trial license.

Vendor detail

Cisco — enterprise stack

Catalyst wireless / Cisco ISE / DNA Center & Catalyst Center / CMX & DNA Spaces

Free sandbox · Multiple mechanisms

Q1 — HOW YOU GET DATA OUT

No single channel — the realistic design combines two layers:

- **ISE RADIUS accounting** — WLC/Catalyst 9800 or switch sends Start/Interim-Update/Stop to ISE (or your own collector) over UDP 1813.
- **ISE MnT REST APIs** (`ActiveSessionsList` / `AuthSessionsList`) — query live/historical sessions on the Monitoring node.
- **ISE pxGrid Session Directory** — real-time pub/sub over STOMP: bulk-download active sessions, then subscribe for live start/update/disconnect.
- **DNA Center / Catalyst Center Assurance Intent API** — REST; AP association, SSID, building/floor site hierarchy.
- **CMX / DNA Spaces** — REST + webhooks for location, presence, zone entry/exit.

Q2 — FORMAT

RADIUS accounting is raw RADIUS AVPs over UDP. ISE MnT/ERS and DNAC/Catalyst Center are JSON (some XML on older ISE) over REST/HTTPS. pxGrid is JSON over STOMP-over-WebSocket/TLS. No protobuf/gRPC anywhere in this stack.

Q3 — FOUR CORE FIELDS

RADIUS/pxGrid alone: clean `User-Name`, `Calling-Station-Id` (MAC), timestamp, and `Acct-Status-Type` state — but location is only a raw NAS-IP/AP MAC, not a human-readable site. DNAC/CMX are the inverse: strong location, weak/no authenticated identity (MAC-keyed) unless joined with ISE.

Design implication: pair RADIUS/pxGrid (identity + state) with DNAC/CMX (readable location_id) — no one API gives all four cleanly.

Q4 — ACCESS REQUIRED

RADIUS: just network access + shared secret, no cloud account. ISE REST: admin account with API access enabled. **pxGrid requires an ISE Plus/Advantage license** — a real gate. DNAC/Catalyst Center Assurance API is generally behind the Advantage license tier (community-sourced, not fully confirmed on an official page). CMX/DNA Spaces are separately licensed.

Free DevNet account unlocks Always-On sandboxes (Catalyst Center) and reservable sandboxes (ISE, including one with a RADIUS simulator) — no hardware purchase needed.

SOURCES

- developer.cisco.com/ … /identity-services-engine
- … /using-api-calls-for-session-management
- developer.cisco.com/docs/pxgrid/session
- … /subscribe-to-session-directory-topic
- developer.cisco.com/docs/catalyst-center
- … /get-overall-client-health
- developer.cisco.com/ … /cisco-cmx location-overview
- developer.cisco.com/docs/sandbox
- ise-support.com – pxGrid licensing

Cisco Meraki — separate, cloud-managed stack

Dashboard API / Scanning & Location API v3 / webhooks — owned by Cisco but architecturally unrelated to ISE/DNAC

Free sandbox | Identity is weak by default

Q1 — HOW YOU GET DATA OUT

- **Dashboard REST API (polling)** — `GET /networks/{id}/clients`, plus `GET /networks/{id}/wireless/clients/{id}/connectivityEvents` for discrete connect/roam/disconnect events.
- **Webhooks** — `client_connectivity` alert fires on connect/disconnect (near-real-time, alerting-shaped rather than a dedicated session stream).
- **Scanning/Location API v3** — Meraki's cloud POSTs ~1 JSON payload/minute/network with per-client RSSI/triangulated x/y observations (needs MR firmware R26+/R27.5+).
- **RADIUS accounting** — supported, but only inside the sign-on/splash-page auth flow; Dashboard is the RADIUS client, not the AP directly.

Q2 — FORMAT

Everything is JSON over HTTPS — REST responses, webhook POST bodies (with a `sharedSecret` field for validation), and Scanning API observation arrays. RADIUS accounting to your own server is standard RADIUS wire format. Dashboard UI also offers manual CSV export, not suited to automation.

Q3 — FOUR CORE FIELDS

Location, timestamp, and state-ish signals are solid. **Identity is the weak link**: the `clients` endpoint's `user` field only populates for 802.1X, SAML/sign-on splash, or Identity PSK auth — open/PSK/MAC-auth SSIDs give MAC + SSID only, no real person.

> Location API v3 reportedly adds a `userId` field (email/username) per a secondary source — unverified against Cisco's primary schema doc, worth confirming before relying on it.

Q4 — ACCESS REQUIRED

API key from a Dashboard admin account (`X-Cisco-Meraki-API-Key` header), inherits that admin's org/network permissions. Rate limit: 10 req/sec/org (burst +10), 100 req/sec per source IP.

Free, always-on Meraki developer sandbox — no hardware purchase needed to explore the API, though it won't generate organic live client traffic; production data needs a real licensed Meraki network.

SOURCES

`developer.cisco.com/meraki/api-v1` `… /get-network-clients`

`developer.cisco.com/meraki/webhooks` `… /webhook-sample-alerts` `developer.cisco.com/meraki/scanning-api`

`documentation.meraki.com — RADIUS & Sign-On Splash`

Aruba — HPE Aruba Networking

Aruba Central (cloud) / ClearPass Policy Manager (NAC/RADIUS) / Analytics & Location Engine

License-gated | Strong field coverage

Q1 — HOW YOU GET DATA OUT

- **Central Monitoring REST** — `GET /monitoring/v1/clients` (and `/v2`, wireless/wired variants) — polling snapshots, not events.
- **Central Streaming API** — secure WebSocket pub/sub (topics: Monitoring, Presence, Location, AppRF, Audit); Monitoring topic emits state-change messages plus 5-minute periodic stats — closest thing to a real session-event stream. Requires an **Advanced-licensed device**.
- **ClearPass RADIUS accounting** — classic Start/Interim-Update/Stop to an external server, vendor-neutral.
- **ClearPass REST** — Insight API and GuestManager `ActiveSession` service (`GET /session`, filterable/sortable by MAC, IP, `acctstarttime` / `acctstoptime`).
- **Aruba ALE** — separate appliance; polling REST + pub/sub streaming for real-time location.

Q2 — FORMAT

JSON over REST for Central and ClearPass/Insight/GuestManager. JSON over WSS for Central Streaming and ALE pub/sub. RADIUS accounting is standard RADIUS AVP wire format.

Q3 — FOUR CORE FIELDS

All four map well: `username` and MAC on Central/CPPM, AP name + site (v2) on Central, x/y on ALE, `Association Time` / `acctstarttime` - `acctstoptime` for timing. **RADIUS accounting's `Acct-Status-Type` is the cleanest fit for session state** — Central's REST is snapshot/poll-based, so start/stop needs diffing unless you use the Streaming API.

> Exact JSON schema for the Streaming API's Monitoring-topic payload wasn't fully confirmed in docs excerpts — verify against Aruba's Postman/OpenAPI spec before building against it.

Q4 — ACCESS REQUIRED

Central: account + API Gateway license, OAuth2 token (2h expiry, 14-day refresh) — **API Gateway must be enabled by an HPE Aruba sales rep**, not self-service. Streaming additionally needs an Advanced-licensed device. ClearPass needs admin access + OAuth2 client credentials, and RADIUS needs a shared secret — **ClearPass itself requires purchased hardware/VM, no public trial found**. A historical 90-day/60-device Central trial exists but current availability for API Gateway specifically is unconfirmed.

SOURCES

`developer.arubanetworks.com/central/streaming-events`

`… /streaming-api-getting-started` `… /api-gateway`

`developer.arubanetworks.com/cppm — getting started`

`arubanetworking.hpe.com/techdocs — ALE, Streaming API, GuestManager`

`arubanetworking.hpe.com — Central free trial`

Ruckus — CommScope

SmartZone controller / RUCKUS One (cloud) / SPoT location

60-day cloud trial | Protobuf/MQTT for streaming

Q1 — HOW YOU GET DATA OUT

- **SmartZone Northbound REST ("Public API")** — `GET /wsg/api/public/v{ver}/query/client` and per-AP/WLAN variants; session-logon (`JSESSIONID`) then poll for current state.
- **Northbound Data Streaming** (formerly SCI) — real push interface: **Google Protobuf over MQTT**, SmartZone publishes, you run the MQTT broker.
- **RADIUS accounting** — SmartZone can proxy/forward standard Start/Interim-Update/Stop to an external RADIUS server.
- **RUCKUS One** (cloud) — REST under Client Management / Events and Alarms.
- **SPoT** — indoor positioning; Push API, also GPB over MQTT (topic pattern `{version}/LOC/SPOT_GPB/{venue_id}/{data_type}`).

Q2 — FORMAT

REST (SmartZone Public API, RUCKUS One) is JSON over HTTPS. Northbound Data Streaming and SPoT Push API are **binary Protobuf over MQTT** — needs `.proto` schema files from Ruckus support, heavier tooling than plain REST/JSON. RADIUS is standard RFC 2866 AVPs.

Q3 — FOUR CORE FIELDS

`apMac`/zone/WLAN ID give strong location; timestamps present everywhere. **Identity depends entirely on auth type** — RADIUS/802.1X gives `userName`, open/PSK gives MAC only. RADIUS accounting's Start/Interim-Update/Stop is the only path with an explicit state enum; the REST query API is snapshot-only unless you diff polls yourself.

Q4 — ACCESS REQUIRED

SmartZone Public API / Northbound Streaming: **owned/licensed SmartZone controller** (physical or virtual) + admin credentials — no public sandbox. RADIUS: just a shared secret and NAS config, no account needed. RUCKUS One: cloud account, client ID/secret → JWT, and a **60-day free trial** exists for cloud API access (real APs still needed for live client data). SPoT access details are unclear — likely a commercial partner account.

SOURCES

- docs.ruckuswireless.com — SZ 5.0 API Guide
- docs.ruckuswireless.com — SZ 7.1.1 API Guide
- docs.commscope.com — Northbound Data Streaming config
- docs.ruckus.cloud/api
- github.com/rksg — SPoT Push API sample client
- ruckusnetworks.com — RUCKUS One trial

Ubiquiti / UniFi

UniFi Network Application / official Network API / Site Manager (cloud) API

No sandbox — needs owned hardware | No identity by default

Q1 — HOW YOU GET DATA OUT

- **Official UniFi Network API** — versioned REST, API key generated in the UniFi OS console UI (`developer.ui.com`), e.g. `GET` connected-clients endpoint.
- **Legacy/unofficial controller API** — `/api/s/{site}/stat/sta` (active clients), `/api/s/{site}/stat/session` (historical) — undocumented by Ubiquiti but widely used via community tools (`unifi-poller`, `node-unifi`).

- **Controller WebSocket event stream** — `wss://{controller}/…/wss/s/{site}/events`, unofficial, pushes real-time join/leave; leave events aren't reliable without a proper disassociation frame.
- **RADIUS accounting** — UniFi gateways act as an 802.1X authenticator against your own external RADIUS server (FreeRADIUS, NPS, etc.) — UniFi itself is not the RADIUS server.
- **Site Manager API** — official cloud REST (`api.ui.com`), aggregates across sites, coarser-grained than per-site Network API.

Q2 — FORMAT

All REST and WebSocket paths are JSON. RADIUS accounting is standard RADIUS AVPs over UDP, not JSON — needs a separate accounting listener, not an HTTP client.

Q3 — FOUR CORE FIELDS

Location (AP MAC, ESSID, site) and timestamps (`first_seen` / `last_seen`) are well-supported everywhere. **Session state is partial** — `stat/sta` is a snapshot (poll-and-diff to infer start/stop), `stat/session` gives historical start/end, WebSocket leave-detection is unreliable.

> Identity is the confirmed weak point: UniFi client records are MAC/device-centric by default, no user_id field. A real identity only appears via a configured guest-portal IdP or 802.1X/RADIUS username — without either, device_mac is the only durable key, breaking the "stable identity across devices" assumption.

Q4 — ACCESS REQUIRED

Official Network API and legacy/unofficial API: local admin credentials/API key on the console, direct network reachability — **requires owning/administering real UniFi hardware or a self-hosted controller**. Site Manager API: a Ubiquiti cloud account + API key, still needs an adopted UniFi console. RADIUS: no Ubiquiti credential at all, just your own RADIUS server.

Confirmed: no public Ubiquiti-hosted sandbox exists, official or otherwise — unlike Cisco DevNet, every path needs owned hardware.

SOURCES

help.ui.com — Official UniFi API

developer.ui.com/network — getting started

developer.ui.com/site-manager — getting started

ubntwiki.com — community API docs (unofficial)

unpoller.com/docs — Unpoller (unofficial)

artofwifi.net — auth method comparison

help.ui.com — configuring RADIUS in UniFi

`help.ui.com — External Hotspot/Guest Auth API`

Juniper Mist

Cloud-managed WiFi, AI-driven (Marvis) / Mist Access Assurance NAC

Sandbox unconfirmed   Strong location (vBLE RTLS)

Q1 — HOW YOU GET DATA OUT

- **REST polling** — `GET /api/v1/sites/{site_id}/stats/clients` (also map-scoped for location-tagged clients).
- **WebSocket streaming** — subscribe-based, mostly stats/location channels (e.g. live client locations on a map); no dedicated raw "client-events" channel confirmed.
- **Webhooks** — topic-based, batched: `client-join`, `client-sessions` (fullest session detail per client-per-AP), plus `location`, `zone`, `nac-accounting`, `device-events`.
- **RADIUS accounting** — standard RFC 2866, strongest for identity + session start/stop, especially under Mist Access Assurance (cloud NAC/802.1X).

Q2 — FORMAT

REST is JSON/HTTPS. WebSocket is a JSON envelope (`{"event":"data","channel": … ,"data":{ … }}`). Webhooks are JSON POST, aggregated/batched per topic rather than one message per event. RADIUS is standard AVPs — needs a translation layer in the adapter.

Q3 — FOUR CORE FIELDS

Location is a genuine strength — vBLE-based indoor RTLS (cm-to-meter accuracy) via `location` / `zone` / `vbeacon` topics. Timestamps are standard throughout. **Session state is only partial** outside RADIUS — `client-join` / `client-sessions` approximate start/update, no clean confirmed stop/disconnect event. **Identity is weak** on plain WiFi stats (MAC-centric, hostname but not username tracked) — real identity requires the separately licensed Access Assurance NAC add-on.

Q4 — ACCESS REQUIRED

Mist cloud org account, API tokens (org-level or user-level) via Organization → Admin → Settings. Rate limit: 5,000 calls/hour/token, higher limits need a Juniper support ticket. **No public free-trial/sandbox org was confirmed** — likely needs a Juniper/Mist partner or demo account. vBLE

RTLS and Access Assurance/NAC are separately licensed and need specific AP hardware (antenna array).

SOURCES

`juniper.net — RESTful API overview` `juniper.net — WebSocket API overview` `api-class.mist.com — Get WiFi Clients` `juniper.net — Webhook messages` `mist.com — Mist RADIUS attributes` `juniper.net — Access Assurance overview` `juniper.net — Location Services overview` `mist.com — API rate limiting`

Huawei

AirEngine APs / iMaster NCE-Campus / eSight (legacy)

Docs gated behind support login | Identity unconfirmed

Q1 — HOW YOU GET DATA OUT

- **iMaster NCE-Campus Northbound REST APIs** — the primary modern path; HTTPS on the controller's northbound floating IP, port 18002. "Terminal Management APIs" expose station (STA) query — closest match to client/session data.
- **RADIUS accounting** — AirEngine APs/WACs act as standard RADIUS clients (auth port 1812, accounting port 1813) — vendor-neutral, standard Start/Interim/Stop.
- **eSight (legacy NMS)** northbound interfaces — REST, SNMP, FTP, and Kafka, but oriented toward alarms/performance/topology, not per-client sessions.
- **Streaming Telemetry** exists on iMaster NCE-CampusInsight (analytics add-on) via gRPC/Protobuf — no evidence it's a per-user-session channel, appears to be infrastructure health metrics only.

Q2 — FORMAT

NCE-Campus NBI is JSON over HTTPS REST, token auth via `/controller/v2/tokens`, TLSv1.2. RADIUS accounting is standard RADIUS AVPs over UDP. eSight is RESTful JSON plus SNMP traps and Kafka streams for other data types. CampusInsight telemetry is Protobuf over gRPC.

Q3 — FOUR CORE FIELDS

Location (AP/SSID association) and timestamp (an explicit "online duration" field) are confirmed in the Terminal Management API. Device MAC is an explicit returned field.

> Identity and session-state coverage could not be fully verified: no explicit "username" field was confirmed in the NBI docs accessible (paywalled beyond snippets) — identity likely has to come from the RADIUS User-Name attribute in the accounting path rather than the REST query. Session state on NBI would need poll-and-diff (online/offline status); no push/webhook mechanism for start/stop was found. RADIUS accounting remains the cleanest 1:1 match to the target enum.

Q4 — ACCESS REQUIRED

NCE-Campus NBI needs an admin-provisioned "Third-party" account with the "Open API Operator" role on a live controller instance. **Most support.huawei.com API doc pages render blank/JS-gated without a Huawei support login** — a real access barrier compared to Cisco/Aruba/Juniper's public docs. A developer portal (devzone.huawei.com, "CloudCampus"/API Studio) suggests free registration is possible for browsing docs and SDKs. One forum/doc snippet mentioned a sandbox supporting up to 200 concurrent users for API debugging, but this could not be independently verified. Real production access likely needs a partner-adjacent trial license via Huawei's ESDP portal.

SOURCES

`support.huawei.com — NCE-Campus API Reference`

`info.support.huawei.com — Terminal Management API`

`support.huawei.com — RESTful API Dev Guide`

`support.huawei.com — Configuring RADIUS AAA (AirEngine X700)`

`support.huawei.com — eSight Northbound Interfaces`

`support.huawei.com — CampusInsight Telemetry`

Compiled from official vendor developer documentation via seven parallel research passes (one per vendor/product line), cross-checked against the `SessionEvent` model in `presence-platform/src/presence_platform/core/models.py`. Fields marked "unconfirmed" indicate the docs available did not settle the question either way, not that the capability doesn't exist — treat those as next things to verify directly (sandbox account, vendor SE, or a Postman collection) before an adapter design depends on them.